// ML-ARENA_CHALLENGE_194 SYS.ORAL_2027-01-06

# DS-Harness : Agent Rule-First avec Fallback LLM

Résolution déterministe ciblée, règles croisées & génération Python supervisée sur Qwen2.5-Coder-1.5B

| MEILLEURE SOUMISSION · V10 | PREMIÈRE SOUMISSION · V1 | FREEZE · FIN DE PROJET | ÉVALUATION ORAL |
|---|---|---|---|
| 97.5 % (117/120) | 51.7 % (62/120) | 2027-01-04 | 2027-01-06 9h–12h |

# Exigences du Challenge ML-Arena

120 TASKS PRIVATE

OBJ.01

## Réponse numérique unique

Métrique stricte avec tolérance décimale ($1.5 \times 10^{-n}$) pour n décimales demandées.

OBJ.02

## Modèle embarqué local

Modèle de ~1.5B paramètres (Qwen2.5-Coder) chargé via `dsh.load_llm`, zéro réseau.

OBJ.03

## Budget temps strict

40 s par lot de 8 tâches (5 s par problème), 15 appels. Un dépassement fait échouer le déploiement.

OBJ.04

## Isolation matérielle

3 vCPUs et 3 GiB de RAM, sans GPU ni possibilité de dépassement.

NO FILES · 45

Arithmétique, pourcentages, croissance, dates, probabilités, problèmes

« A savings account starts at 5000 euros and earns 4% per year… How much interest has been earned after 4 years? »

TABLES · 50

Statistiques, filtres, classements de groupes, jointures, colonnes dérivées

« Compute the mean amount borrowed for each branch. What is the smallest of these values? » (CSV séparé par « ; », virgule décimale)

FITS · 25

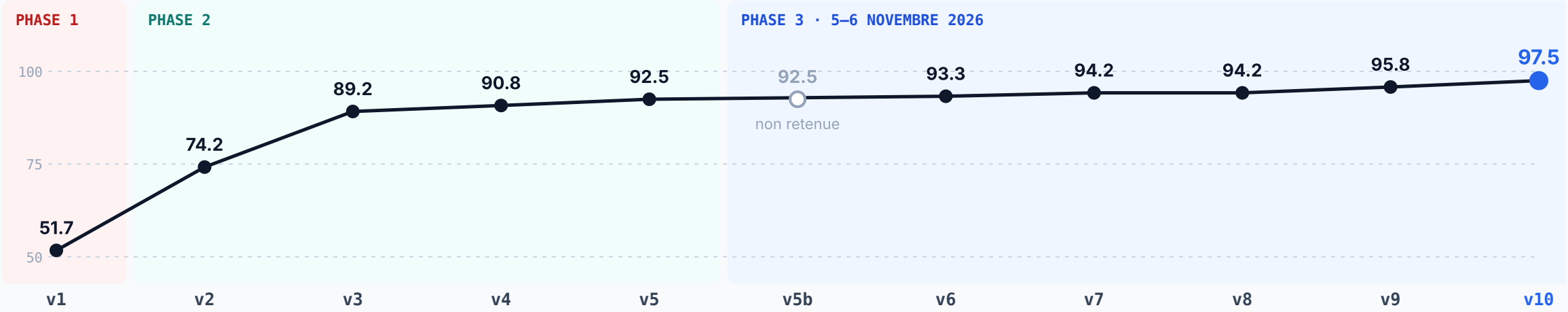
Régression linéaire ou logistique sur `train.csv`

« Fit an OLS regression of the monthly rent on the living area, rooms, floor… What is the intercept? »

[ SYS.02 // HISTORIQUE DU PROJET ]

# 10 versions soumises, en 3 phases

SCORE PRIVÉ (%) PAR VERSION




**PHASE 1 · LE LLM FAIT TOUT**

| | | |
|---|---|---|
| **v1** | 51.7 | Le LLM écrit et exécute le code Python de chaque tâche. |

**PHASE 2 · RÈGLES D'ABORD, LLM EN REPLI**

| | | |
|---|---|---|
| **v2** | 74.2 | Règles de fit, correctifs d'exécution du code généré |
| **v3** | 89.2 | Premières règles sans fichier et tables |
| **v4** | 90.8 | Règles sans fichier étendues (+2 tâches) |
| **v5** | 92.5 | Règles de tables affinées (+2) : `stage3.py` |

**PHASE 3 · ÉLARGIR ET CONTRÔLER LES RÈGLES**

| | | |
|---|---|---|
| **v5b** | 92.5 | Réécriture indépendante de toutes les règles ; tables 84 %, non retenue |
| **v6** | 93.3 | Tables de v5, sans fichier et fits de v5b |
| **v7** | 94.2 | + 21 types de tables absents du dev, veto |
| **v8** | 94.2 | + gabarits sans fichier, milliers, contrôles de forme |
| **v9** | 95.8 | + indices du parseur dans le prompt, 8 échantillons |
| **v10** | 97.5 | + croisement des deux jeux de règles de tables |

**Vocabulaire des slides suivantes :** « règles historiques » = règles écrites de v2 à v5 (`stage3.py`) ; « règles réécrites » = jeu indépendant écrit en v5b, corrigé de v7 à v10.

[ SYS.03 // RATIONALE DE DESIGN ]

# Pourquoi la Stratégie Rule-First ?

LIMITES

## Problèmes du pur-LLM 1.5B

- Hallucinations fréquentes sur les calculs exacts
- Erreurs de syntaxe Python sous pression
- Latence élevée sur CPU, budget de 5 s par tâche
- Notre v1 (LLM seul) : 51.7 % au privé ; référence du challenge : 53.3 % sur le dev

VERSUS →

AVANTAGES

## Atouts de DS-Harness

- Réponse exacte dès qu'un gabarit est reconnu
- Exécution en quelques millisecondes
- Budget temps préservé pour les cas ouverts
- **Une règle s'abstient si elle n'est pas sûre**

**Leçon de la phase 3 :** une règle fausse est pire qu'une abstention. Le LLM rattrape une abstention, jamais une règle qui se trompe.

DÉMARCHE, DE V1 À V10

1 · DIAGNOSTIQUER
Familles et formulations qui échouent

›

2 · UNE HYPOTHÈSE
Un seul changement par soumission

›

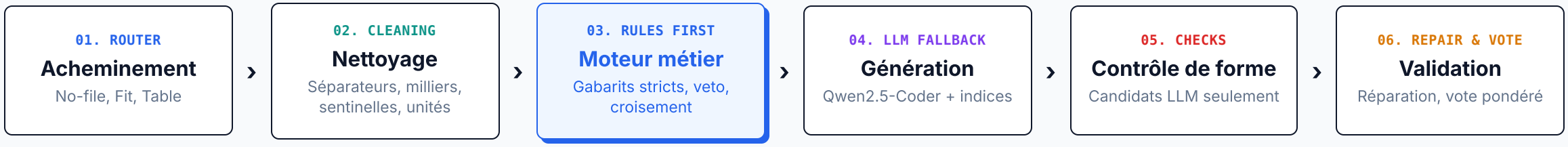
3 · VÉRIFIER HORS LIGNE
Dev, tâches générées, non-régression

›

4 · LE PRIVÉ TRANCHE
On garde seulement ce qui gagne

[ SYS.04 // FLUX DE TRAITEMENT ]

# Architecture globale du pipeline




**CHEMIN DES TABLES DANS LA VERSION FINALE (V10)**

**1 · GABARITS ÉTROITS**
21 types absents du dev : IQR, top-k, mode, négations, A ou B…

**2 · VETO**
Constructions mal lues par les règles historiques → directement au LLM

**3 · RÈGLES HISTORIQUES**
Code de v2–v5 (`stage3.py`) conservé à l'identique

**4 · CROISEMENT**
Désaccord avec les règles réécrites → la réponse réécrite gagne

**5 · LLM**
Prompts de la phase 2 + indices du parseur, 8 échantillons si ≤ 4 tâches

[ SYS.05 // MOTEUR DÉTERMINISTE ]

# Périmètre d'Action des Règles

FAMILLE 01 · 25/25 PRIVÉ (V10)

## Fits (20.8 %)

Cible = colonne de `train.csv` absente de `test.csv`. Régression linéaire ou logistique sans régularisation, tolérante aux NaN. Prédiction, $R^2$, coefficient, odds ratio, moyenne prédite, comptages.

FAMILLE 02 · 48/50 PRIVÉ (V10)

## Tables (41.7 %)

Jointures avec clés normalisées, filtres de dates et d'opérateurs, agrégats par groupe. 21 gabarits étroits ajoutés en v7, deux jeux de règles indépendants croisés en v10.

FAMILLE 03 · 44/45 PRIVÉ (V10)

## No-files (37.5 %)

PPCM, logarithmes, systèmes linéaires, probabilités, intérêts composés, dates. Environ 130 gabarits ajoutés en phase 3, en trois vagues, chacun strict : il ne répond que si toute la question correspond.

Les règles répondent seules aux 180 tâches du dev public, sans appel au LLM. Ce score est attendu par construction : le privé reste la seule mesure.

[ SYS.06 // RÈGLES CROISÉES — AJOUTÉ EN v10 ]

# Deux jeux de règles, une réponse

DIAGNOSTIC

## Les règles historiques se trompent parfois sans le savoir

| | |
|---|---|
| Variantes proches du dev | 11 / 500 faux |
| Types absents du dev | 50 / 420 faux |

- Mesuré sur des tâches générées, avec vérité connue
- Écart interquartile calculé comme une étendue
- « distinct carriers per depot » compté en lignes
- « not of the Lyon store » lu « of the Lyon store »

RÈGLE DE DÉCISION

Les règles historiques et les règles réécrites répondent et divergent → **la réponse réécrite l'emporte**. Sinon la réponse historique est conservée.

LIMITE VOLONTAIRE

Quand les règles historiques s'abstiennent, la tâche va au LLM avec indices : y appliquer les règles réécrites a coûté 5 tables au privé (86 %).

RÉSULTAT PRIVÉ, V9 → V10

**Tables 92 % → 96 %**

+2 tâches, score global 95.8 % → 97.5 %

# Supervision du LLM Qwen2.5-Coder

PROMPTING

## Contexte réduit + indices

- Modèle `Qwen2.5-Coder-1.5B`
- Schéma exact, dictionnaire, un exemple guidé
- **Indices du parseur** : colonnes résolues, population et lignes comptées d'une proportion, filtres

EXÉCUTION

## Boucle auto-correctrice

- Thread surveillé, `max_time` sur chaque génération
- Réparation à partir du traceback
- Vote pondéré : glouton 1.5, échantillon 1.0
- **8 échantillons** quand ≤ 4 tâches du lot restent ouvertes

CONTRÔLES

## Forme de la réponse

- Fraction, probabilité dans [0, 1]
- Comptage entier, rang ≥ 1, jour 1–7
- Date YYYYMMDD valide, dispersion ≥ 0
- Jamais appliqué aux réponses des règles

**EFFET MESURÉ AU PRIVÉ**

Indices + 8 échantillons, v8 → v9 : tables 88 % → 92 % (94.2 → 95.8 %)

**SÉCURITÉ DES CONTRÔLES**

0 vraie réponse rejetée sur 9 811 tâches générées

[ SYS.08 // CORRECTIFS INGÉNIERIE ]

# Robustesse de l'Environnement

PATCH 01 · PHASE 2

### Survie de read_csv

Le `read_csv` nettoyé survit à la réimportation de pandas par le code généré.

PATCH 02 · PHASE 2

### SmartFrame

Résolution des fautes de frappe et des variations de noms de colonnes.

PATCH 03 · PHASE 2

### Budget temps interne

Génération bornée à 29 s sur 40 s pour éviter l'échec global du lot.

PATCH 04 · PHASE 2

### Harmonisation des clés

Normalisation des préfixes et zéros initiaux avant les jointures.

PATCH 05 · PHASE 3

### Champs contenant « of » et apostrophes

`length of stay` n'est plus coupé en « stay » ; `company's sites` n'ouvre plus de citation.

PATCH 06 · PHASE 3

### Opérateurs

`no less than 5` → ≥ 5 ; `(between 0 and 1)` après « fraction » n'est plus un filtre.

PATCH 07 · PHASE 3

### Séparateurs de milliers

`24,000` et `1 tonne = 1,000 kg` ne sont plus tronqués à la virgule.

PATCH 08 · PHASE 3

### Intérêts seuls

« How much interest has been earned » ne renvoie plus le solde final.

[ EVAL.01 // PROTOCOLE ]

# Dev Set vs Private Leaderboard

DIAGNOSTIC SEULEMENT

### Public dev set

180 tâches publiques utilisées pour concevoir et valider les règles déterministes.

**100 % attendu par construction (biais de conception).**

FILET DE NON-RÉGRESSION

### Tâches générées

Générateurs des 7 domaines de tables, 5 domaines de fits, 9 catégories no-file, avec les pièges du dev et une vérité calculée à part.

**Utile, mais ne couvre pas toutes les formulations privées.**

MESURE OFFICIELLE

### Private set (masqué)

120 tâches masquées évaluant la capacité réelle de généralisation de l'agent.

**Seule mesure valide pour le classement, et seule mesure du chemin LLM (pas de GPU).**

[ EVAL.02 // CONTRÔLES HORS LIGNE ]

# Tester sans le modèle

| CONTRÔLE | RÉSULTAT |
|---|---|
| Dev, règles seules | 180/180 |
| Dev, `solve()` complet avec LLM factice | 180/180 · ≤ 1.5 s/lot |
| Tables générées (types dev et nouveaux) | 1 548/1 550 · 0 faux |
| 21 nouveaux types de tables, deux tirages | 420/420 ×2 |
| No-files générés, trois vagues | 100 % · 0 faux |
| Paraphrases du dev : tables / no-files / fits | 765/775 · 542 · 244 |
| Non-régression des no-files, v8 contre v7 (6 505 tâches) | 0 régression |
| Contrôles de forme sur vraies réponses | 0 rejet / 9 811 |

ESTIMATION GOOD-TURING

## ≈ 1/3

des no-files privées viendraient de sous-gabarits absents du dev. D'où des vagues de gabarits larges plutôt que la chasse à une seule tâche.

**PRINCIPE**

Vérité calculée sur les données propres, indépendamment de l'agent ; comparaison stricte des réponses et des prompts entre versions.

[ EVAL.03 // TRAJECTOIRE ]

# Score privé de chaque version

| VERSION | CE QUI CHANGE | SANS FICHIER | TABLES | FITS | TÂCHES | SCORE |
|---|---|---|---|---|---|---|
| PHASE 1 · LE LLM FAIT TOUT | | | | | | |
| v1 | Agent LLM de base | 75.6 | 40.0 | 32.0 | 62/120 | **51.7** |
| PHASE 2 · RÈGLES D'ABORD, LLM EN REPLI | | | | | | |
| v2 | Règles de fit, correctifs d'exécution | 77.8 | 60.0 | 96.0 | 89/120 | **74.2** |
| v3 | Règles sans fichier et tables | 91.1 | 84.0 | 96.0 | 107/120 | **89.2** |
| v4 | Règles sans fichier étendues | 95.6 | 84.0 | 96.0 | 109/120 | **90.8** |
| v5 | Règles de tables affinées (stage3.py) | 95.6 | 88.0 | 96.0 | 111/120 | **92.5** |
| PHASE 3 · ÉLARGIR ET CONTRÔLER LES RÈGLES (5–6 NOVEMBRE 2026) | | | | | | |
| v5b | Réécriture de toutes les règles (non retenue) | 97.8 | 84.0 | 100 | 111/120 | **92.5** |
| v6 | Tables de v5, autres familles de v5b | 97.8 | 86.0 | 100 | 112/120 | **93.3** |
| v7 | + 21 types de tables, veto | 97.8 | 88.0 | 100 | 113/120 | **94.2** |
| v8 | + gabarits sans fichier, milliers, contrôles | 97.8 | 88.0 | 100 | 113/120 | **94.2** |
| v9 | + indices du parseur, 8 échantillons | 97.8 | 92.0 | 100 | 115/120 | **95.8** |
| **v10** | **+ croisement des règles de tables** | **97.8** | **96.0** | **100** | **117/120** | **97.5** |

Scores en % sur les 120 tâches privées (45 sans fichier, 50 tables, 25 fits) ; une tâche = 0.83 point. Référence du challenge : 53.3 % sur le dev (Qwen2.5-Coder seul).

[ EVAL.04 // DÉTAIL PAR FAMILLE ]

# Scores v10 sur le Private Set

FITS (25 TASKS)

**100%**

25 / 25 RÉUSSIES

v1 32.0 % → v5 96.0 % → **v10 100 %**

NO-FILES (45 TASKS)

**97.8%**

44 / 45 RÉUSSIES

v1 75.6 % → v5 95.6 % → **v10 97.8 %**

TABLES (50 TASKS)

**96.0%**

48 / 50 RÉUSSIES

v1 40.0 % → v5 88.0 % → **v10 96.0 %**

SOLUTION RETENUE · AGENT/AGENT_V10.PY

Règles historiques (v2–v5) + 21 gabarits de tables + veto + croisement avec les règles réécrites ; LLM Qwen2.5-Coder avec indices du parseur, 8 échantillons et contrôles de forme. **97.5 % (117/120).**

Depuis v1 : +55 tâches. Sur la phase 3 (v5 → v10) : +1 fit, +1 sans fichier, +4 tables (indices et 8 échantillons +2, croisement +2).

[ EVAL.05 // RÉSULTATS NÉGATIFS ]

# Ce qui n'a pas marché, et ce qu'on en tire

| ESSAI, AJOUTÉ À V9 OU V10 ET SOUMIS UNE FOIS | SCORE |
|---|---|
| Vote pondéré par l'usage des colonnes du parseur | 95.8 |
| 12 échantillons, ≤ 6 tâches, second tour | 95.8 |
| Indices pour les no-files | 95.8 |
| Le LLM arbitre entre les deux jeux de règles | 96.7 |
| Règles réécrites aussi là où les règles historiques s'abstiennent | 93.3 |
| Règles réécrites partout, avant correction (v5b) | tables 84 |

**LEÇON 1**

Même score sur cinq variantes côté LLM → les erreurs restantes étaient **déterministes**.

**LEÇON 2**

Une règle générique se trompe sur des formulations privées que le générateur ne couvre pas.

**LEÇON 3**

Le LLM avec indices bat les règles génériques sur les formulations inédites.

**LEÇON 4**

Bruit de 1 à 2 tâches entre soumissions : seuls v7, v9 et v10 en sortent clairement.

# Analyse des 3 Tâches Manquées

RÉPARTITION DES ÉCHECS

## Private set loss (3/120)

- **2 tables** : inchangées par toutes les variantes côté LLM
- **1 sans fichier** : 44/45 sans changement depuis v5b, malgré trois vagues de gabarits
- **0 fit** : famille complète
- Erreurs par tâche non publiées : non inspectables

LIMITES IDENTIFIÉES

## Ce qu'il faut dire honnêtement

- `LAST_ERR` non réinitialisé entre candidats LLM (code de la phase 2, conservé)
- Exception sur `.round()` appliqué aux floats natifs
- Phase 3 : v6–v10 retenues parmi ~12 soumissions sur le même privé
- Chaque version mesurée une seule fois (bruit de 1 à 2 tâches)
- Chemin LLM jamais mesuré hors ligne (pas de GPU)

// 05_PERSPECTIVES_AND_CLOSE

# Recommandations & Étape Suivante

### 01. MESURE LLM HORS LIGNE

Notebook GPU avec les règles coupées (`DSH_RULES_OFF`) pour comparer les réglages du LLM.

### 02. AUDITER LES RÈGLES HÉRITÉES

Croiser systématiquement deux implémentations : le désaccord révèle les erreurs silencieuses.

### 03. LE PRIVÉ RESTE L'ARBITRE

Générateurs comme filet de non-régression ; une tâche = 0.83 point, ne pas sur-interpréter.

**97.5 %**
au private leaderboard (117/120)

**+55**
tâches réussies de v1 (62) à v10 (117)

**0 faux**
sur 1 550 tables générées (règles réécrites)

**3**
tâches privées encore manquées

## Merci pour votre attention — Place aux questions (10 min)